

Complessità Computazionale

Table of contents

Complessità computazionale	1
Complessità computazionale	1
Complessità in tempo	1
Complessità in spazio	3
Esempi complessità computazionale	4
Esempi complessità computazionale	6
Dipendenza dai <i>valori</i> di input	10
Complessità computazionale: casi di analisi	11
Esempio: i bambini dispettosi	12
Soluzione 1 - variante	14
Stima complessità computazionale	14
Stima complessità computazionale	16
Ottimizzazione	19

Complessità computazionale

Complessità computazionale

La complessità computazionale è lo studio di quanto “costa” in termini di risorse eseguire un algoritmo, in relazione alla dimensione dell’input. Per misurare l’efficienza di un algoritmo in maniera univoca bisogna definire una metrica indipendente dalle tecnologie utilizzate, altrimenti uno stesso algoritmo potrebbe avere efficienza diversa a seconda della tecnologia sulla quale è eseguito.

Complessità in tempo

- **Definizione informale** È la misura del numero di passi elementari (operazioni basilari, ad es. confronti, assegnazioni) che un algoritmo compie in funzione della dimensione n dell’input.

Si misura il tempo in passi, indipendentemente dal tempo necessario per eseguire ciascun passo. L'ipotesi è che ciascun passo 'elementare' abbia lo stesso costo costante

-
- **Formalizzazione** Si definisce una funzione $T(n)$ che, dato n , restituisce il numero massimo (o medio, a seconda dell'analisi) di operazioni eseguite dall'algoritmo su ogni input di dimensione n .
 - **Notazione asintotica** Per descrivere la crescita di $T(n)$ si usa la notazione **Big-O**:

$T(n) = O(f(n))$ significa che, per n sufficientemente grande, il numero di passi dell'algoritmo è al più proporzionale a $f(n)$.

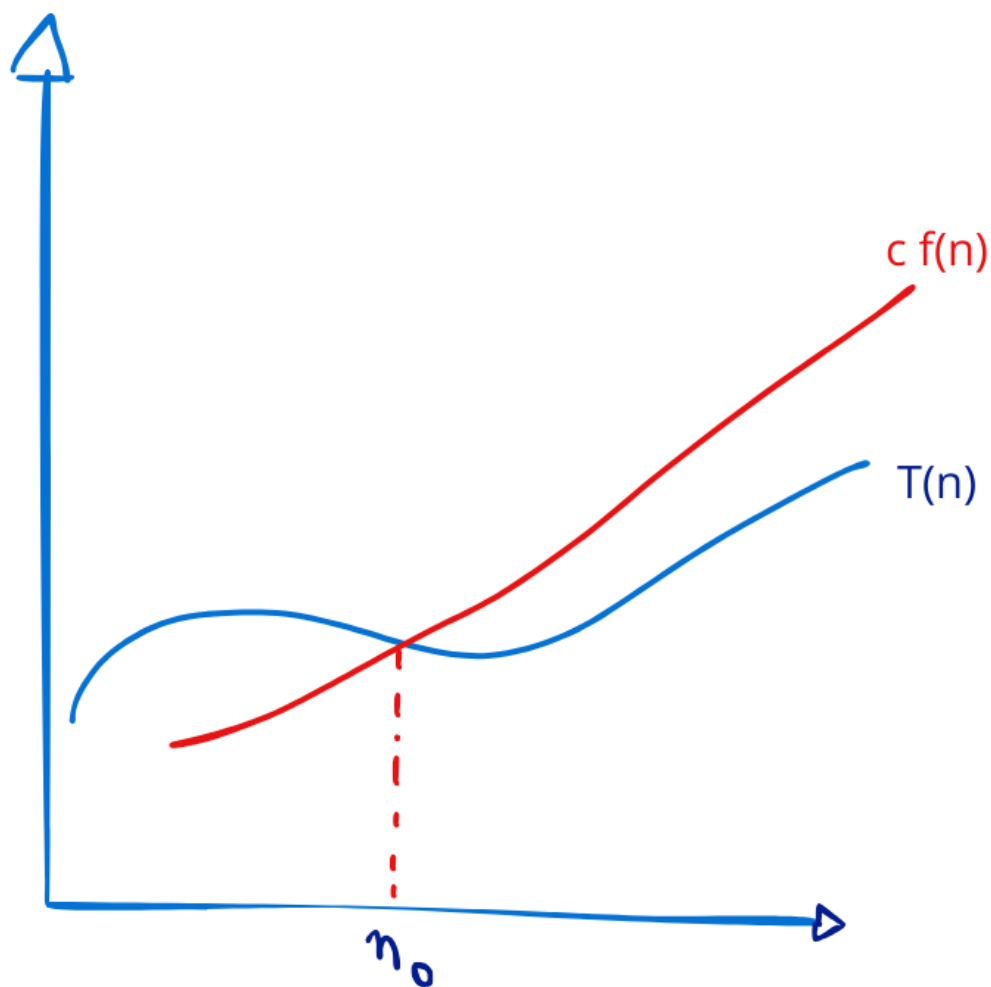
$$T(n) = O(f(n))$$

se $\exists(n_0, c)$ tali che

$$0 \leq T(n) \leq c \cdot f(n)$$

$$\forall n > n_0$$

La funzione $T(n)$ da un certo punto in poi cresce al più come $f(n)$. Diremo che l'algoritmo ha complessità $O(f(n))$, o dell'ordine di $f(n)$



Ci sono altre notazioni asintotiche importanti: notazione Ω (limite asintotico *inferiore*), notazione Θ (limite asintotico *inferiore e superiore*), o , ω .

Esempio: $T(n) = 3n^2 - 4n + 1000$

Da un certo n_0 in poi, $T(n) \leq c \cdot n^2$. Dunque $T(n) = O(n^2)$. L'algoritmo ha complessità *di tempo* $O(n^2)$.

Complessità in spazio

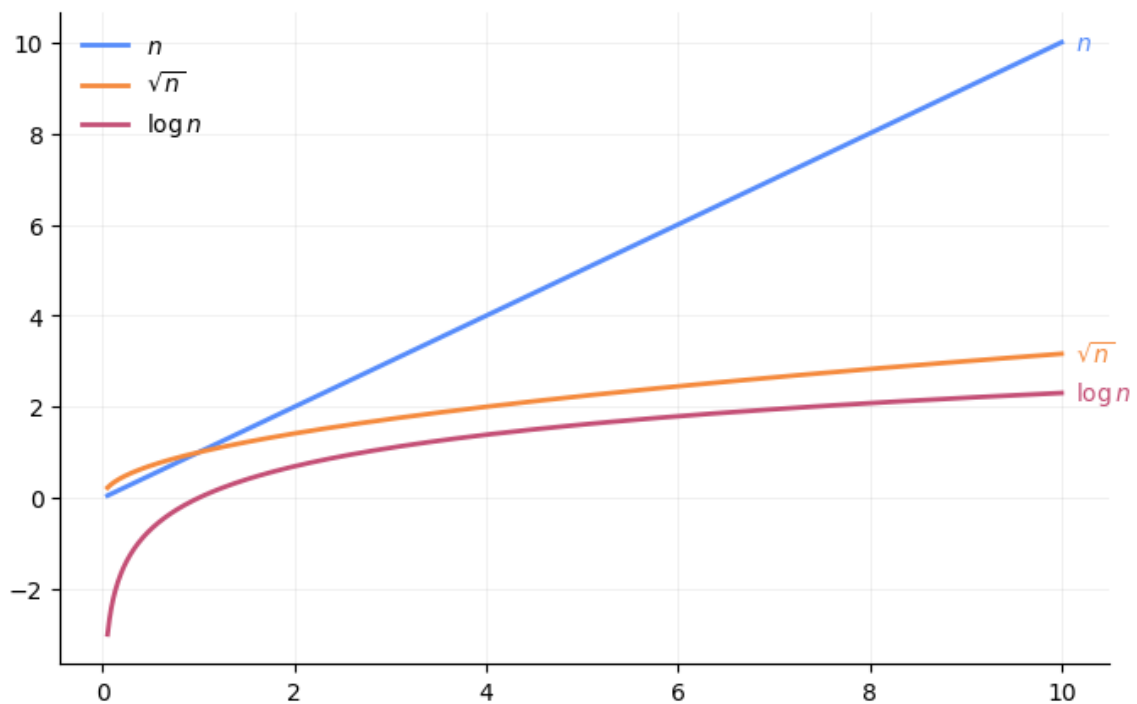
- **Definizione informale** È la misura della quantità di memoria aggiuntiva, oltre allo spazio richiesto per memorizzare l'input, necessaria durante l'esecuzione dell'algoritmo, in funzione della dimensione n .

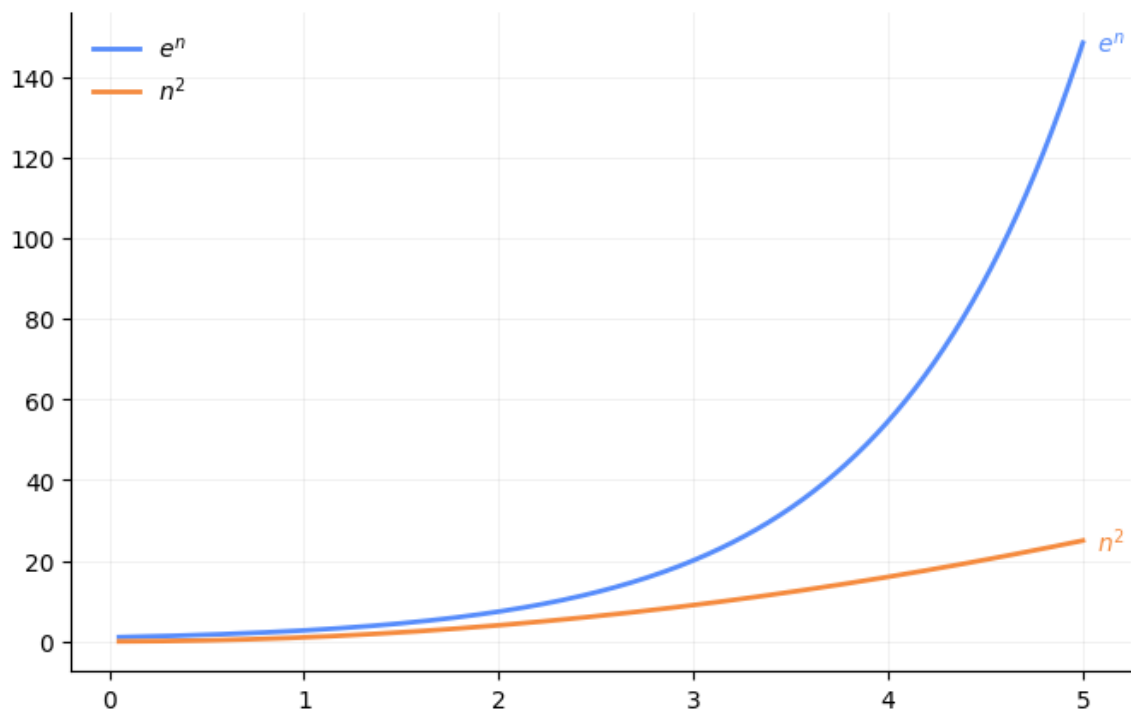
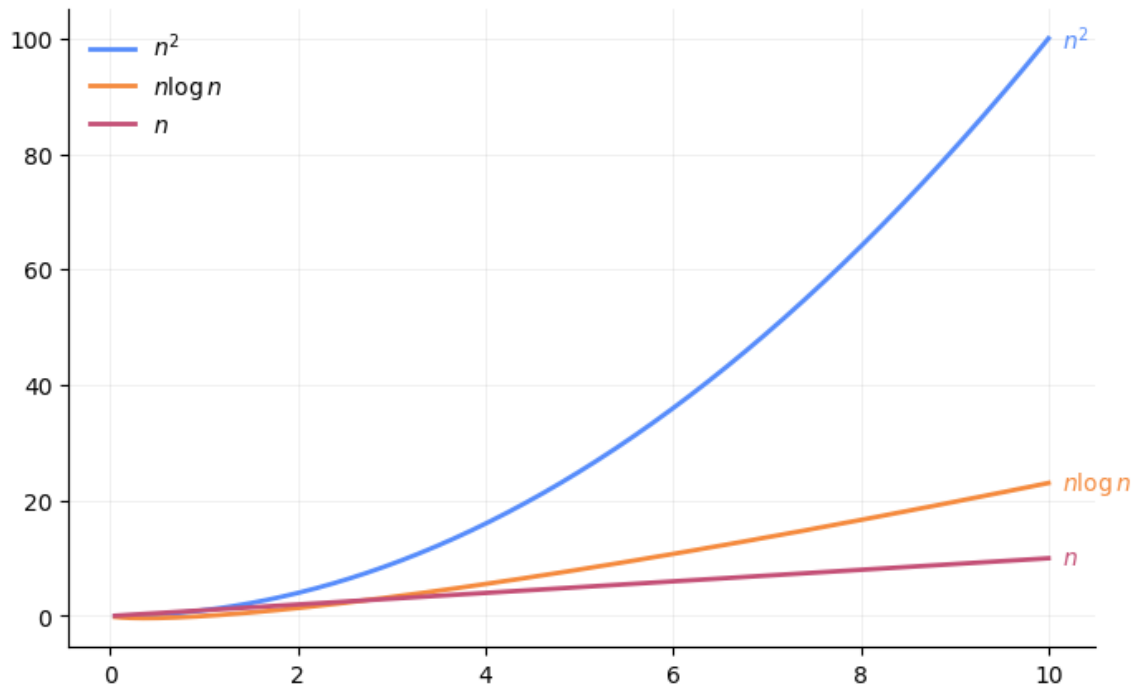
Si usa la notazione Big-O analogamente a quanto visto per la complessità di tempo.

Per il calcolo della complessità in spazio occorre differenziare anche tra Spazio input e spazio ausiliario

- **Spazio input:** memoria necessaria solo per immagazzinare l'input; di solito non si conta nella complessità spaziale.
- **Spazio ausiliario:** memoria extra che l'algoritmo richiede *in aggiunta allo spazio input*; è ciò che definisce realmente la complessità in spazio.

Esempi complessità computazionale





Esempi complessità computazionale

```
// c
i=0;
while (i<n){
    i=i+1;
}
```

```
# python
i = 0
while i<n:
    i=i+1
```

- Assegnamento esterno al ciclo: 1
- Numero di test: $n+1$
- Iterazioni totali: n
- Computazioni interne al ciclo: 1 per ogni iterazione

$$f(n) = 1 + (n + 1) + n = 2 \cdot n + 2$$

$$f(n) = O(n)$$

```
// c
i=0;
while (i<n){
    // ... k assegnamenti
}
```

```
python
i = 0
while i<n:
    // ... k assegnamenti
    i=i+1
```

- Assegnamento esterno al ciclo: 1
- Numero di test: $n+1$

- Iterazioni totali: n
- Computazioni interne al ciclo: k per ogni iterazione

$$f(n) = 1 + (n + 1) + k \cdot n = (k + 1) \cdot n + 2$$

$$f(n) = O(n)$$

```
// c
i=0;
while (i<n){
    // ... k assegnamenti

    i = i+2;
}
```

```
# python
i = 0
while i<n:
    // ... k assegnamenti
    i=i+2
```

- Assegnamento esterno al ciclo: 1
- Numero di test: $n/2 + 1$
- Iterazioni totali: $n/2$
- Computazioni interne al ciclo: k per ogni iterazione

$$f(n) = 1 + (n/2 + 1) + k \cdot n/2 = (k + 1) \cdot n/2 + 2$$

$$f(n) = O(n)$$

```
// c
i=0;
while (i<n){
    for (j=0; j<n; j++){
        // ... k assegnamenti
    }
}
```

```
# python
i = 0
while i < n:
    for j in range(n):
        // ... k assegnamenti
```

- Assegnamento esterno al ciclo: 1
- **while**: Numero di test: $n+1$; Iterazioni totali: n
- **for**: Numero di test: $n+1$; Iterazioni totali: n
- Computazioni interne al ciclo: k per ogni iterazione

$$f(n) = 1 + (n + 1) + n \cdot (n + 1) + k \cdot n^2 = (k + 1) \cdot n^2 + 2 \cdot n + 2$$

$$f(n) = O(n^2)$$

Un solo ciclo FOR, ma complessità $O(n^2)$

```
# python
n = 100
i = 0
j = 0
count = 0

while i < n:
    count += 1
    j += 1
    if j == n:
        j = 0
        i += 1

print("Totale operazioni:", count)
```

$$f(n) = O(n^2)$$

Un solo ciclo FOR, ma complessità $O(\lg_2 n)$

```
# python

i = 1024
count = 0

while i > 1:
    count += 1
    i = int(i/2)
print("Totale operazioni:", count)
```

Ad ogni iterazione il valore viene dimezzato.

Dopo:

- 1 iterazione: $\frac{n}{2}$
- 2 iterazioni: $\frac{n}{2^2}$
- k iterazioni: $\frac{n}{2^k}$

Il ciclo termina quando il valore diventa 1:

$$\frac{n}{2^k} = 1$$

Quindi

$$k = \log_2(n)$$

$$f(n) = O(\lg_2 n)$$

Un solo ciclo FOR, ma complessità $O(n \cdot \lg_2 n)$

```
# python

n = 100
i = 0
j = 1
count = 0

while i < n:
    count += 1
    j = j * 2
    if j >= n:
```

```

        j = 0
        i += 1

print("Totale operazioni:", count)

```

j raddoppia fino ad arrivare ad n , poi viene resettato. Occorrono dunque $\lg_2 n$ iterazioni per arrivare a $j \geq n$.

Il tutto viene ripetuto n volte

Complessità:

$$f(n) = O(n \cdot \lg_2 n)$$

Dipendenza dai *valori* di input

Il numero di passi può dipendere anche dal valore dell'input, non solo dalla dimensione

```

// c
//Cerco l'elemento target con ricerca sequenziale

A={3,4,1,2,0}
s=5;
target=1
i=0;
while(i<s && A[i] != target)
    i = i+1;

```

```

# Python
## Cerco l'elemento target con ricerca sequenziale

A = [3,4,1,2,0]
target = 1

for i in range(len(A)):
    if A[i] == target:
        print ('trovato in posizione', i);
        break;
else:
    print('non trovato')

```

Assegnazioni esterne al ciclo: 4

while (o for): numero di test: 3; iterazioni: 2

Computazioni interne al ciclo: 1 per ogni iterazione

Passi base: 9

Cambiando il `target` il numero di cicli varia

Complessità computazionale: casi di analisi

- **Caso peggiore (worst-case):** numero massimo di passi su qualunque input di dimensione n
- **Caso migliore (best-case):** numero minimo di passi su qualunque input di dimensione n
- **Caso medio (average-case):** valore atteso del numero di passi, tenendo conto dei possibili input. Sarebbe il caso più utile da analizzare perché fornisce un **reale indicatore della complessità dell'algoritmo**.

Per definire un valore “medio” bisogna sapere con quale probabilità ciascun possibile input si presenta. Nella pratica è raro conoscere a priori la distribuzione reale dei dati. Senza un modello di distribuzione affidabile, qualsiasi stima rischia di essere poco significativa.

Inoltre molti algoritmi hanno fasi il cui costo dipende da interazioni non banali tra sotto-strutture dei dati. Analizzare statisticamente tutte le possibili interazioni richiede tecniche avanzate (catene di Markov, equazioni di ricorrenza con termini medi, analisi probabilistica), che possono diventare molto complesse anche per algoritmi apparentemente semplici.

In pratica: spesso ci affidiamo al caso peggiore perché

- da un punto di vista ingegneristico è quello che ci tutela maggiormente
- molto spesso il caso medio è dello stesso ordine di grandezza

Esempio: i bambini dispettosi

In un condominio viene posizionato un secchio d'acqua per piano per poter effettuare le pulizie. In ciascun piano abita un bambino dispettoso che agisce come segue.

- Il bambino al piano i controlla i secchi del piano i e di tutti i **piani multipli di i** . Se il secchio è pieno lo svuota. Se il secchio è vuoto lo riempie.
- I bambini agiscono in sequenza secondo l'ordine del piano: il bambino del piano 1 esamina e 'cambia stato' ai secchi del piano 1, 2, 3...; il bambino del piano 2 esamina e 'cambia stato' ai secchi del piano 2, 4, 6,..., ecc

Consideriamo ora un palazzo di n piani, con i secchi inizialmente vuoti. **Quando tutti i bambini hanno svolto il loro compito, quanti secchi in tutto saranno pieni?**

-
1. Scrivere una funzione che riceve in ingresso il numero di piani del palazzo e rende in uscita il **numero di secchi pieni** al termine del procedimento.
 - Rappresentate i secchi mediante una lista (in Python) o un array (in C). Per semplicità, per avere corrispondenza tra i piani e gli elementi della struttura dati, possiamo considerare solo gli elementi dall'indice 1 in poi, ignorando l'elemento di indice 0.
 - Rappresentiamo con il valore 0 il secchio vuoto e con il valore 1 il secchio pieno.
 2. Stimare la complessità computazionale della funzione.
 3. ragionare sul problema per capire se esiste una **soluzione alternativa che consente di abbattere la complessità computazionale**
-

In Python si può creare una lista di $(n + 1)$ zeri in questo modo

```
lista = [0]*(n+1)
```

In c (c99 o successivi) se voglio creare un array inizializzato a 0 (per esempio all'interno di una funzione) con lunghezza definita dall'utente devo invece inizializzare ciascun elemento dell'array:

```
int f(int n) {
    int arr[n + 1];
    for (int i = 0; i <= n; i++) {
        arr[i] = 0;
    };
    // ...
}
```

```
    return ...  
};
```

Soluzione 1 (Python)

```
def azione_bambini(n):  
    """dato il numero di piani,  
    rende la lista di secchi pieni e vuoti.  
    Si suppone che all'inizio i secchi siano tutti vuoti.  
    Il piano in posizione 0 viene ignorato"""  
    lista_secchi= [0]* (n+1)  
    for i_bambino in range(1,n+1):  
        # il bambino controlla il secchio al proprio piano  
        # e ai piani multipli  
        for i_piano in range(i_bambino,n+1):  
            if i_piano % i_bambino ==0:  
                # agisco solo sui piani 'multipli'  
                if lista_secchi[i_piano]==0:  
                    lista_secchi[i_piano] = 1  
            else:  
                lista_secchi[i_piano] = 0  
    return lista_secchi  
  
def conta_secchi_pieni(lista_secchi):  
    n_secchi_pieni = 0  
    for secchio in lista_secchi:  
        n_secchi_pieni = n_secchi_pieni + secchio  
    return n_secchi_pieni  
  
#---  
n=10000  
lista_secchi = azione_bambini(n)  
n_pieni= conta_secchi_pieni(lista_secchi)  
print(n_pieni)
```

Soluzione 1 - variante

La funzione `conta_secchi_pieni` richiama al suo interno la funzione `azione_bambini`.

Il programma chiamante usa solo `conta_secchi_pieni`.

```
def conta_secchi_pieni(n):
    lista_secchi = azione_bambini(n)
    n_secchi_pieni = 0
    for secchio in lista_secchi:
        n_secchi_pieni = n_secchi_pieni + secchio
    return n_secchi_pieni

#---
n=10000
n_pieni= conta_secchi_pieni(n)
print(n_pieni)
```

Stima complessità computazionale

- Ciclo esterno da 1 a n (variabile `i_bambino`)
- Ciclo interno da `i_bambino` a n

Per il bambino 1, il ciclo interno esegue n iterazioni

Per il bambino 2, il ciclo interno esegue $n - 1$ iterazioni

...

Per il bambino k , il ciclo interno esegue $n - k + 1$ iterazioni

In tutto le operazioni sono

$$\sum_{k=1}^n (n - k + 1) = \frac{n(n-1)}{2}$$

Complessità computazionale $O(n^2)$

-
- Attenzione: è la stessa complessità che avrei per

```
for i in range(n):
    for j in range(n):
        ...
```

Il fatto che il bambino ‘salta’ tutti i piani precedenti il proprio *non* fa risparmiare iterazioni (asintoticamente).

Soluzione 2 (Python)

Modifico il FOR interno in modo che l’indice del piano visitato sia un indice ‘multiplo’ del valore del bambino.

```
# for i_piano in range(i_bambino,n+1):
for i_piano in range(i_bambino,n+1, i_bambino):
    # if i_piano % i_bambino ==0:
    # agisco solo sui piani 'multipli'
    if lista_secchi[i_piano]==0:
        lista_secchi[i_piano] = 1
    else:
        lista_secchi[i_piano] = 0
```

In questo modo genero solo gli indici per i piani ‘giusti’

```
def azione_bambini(n):
    """dato il numero di piani,
    rende la lista di secchi pieni e vuoti.
    Si suppone che all'inizio i secchi siano tutti vuoti.
    Il piano in posizione 0 viene ignorato"""
    lista_secchi= [0]* (n+1)
    for i_bambino in range(1,n+1):
        # il bambino controlla il secchio al proprio piano
        # e ai piani multipli
        # for i_piano in range(i_bambino,n+1):
        for i_piano in range(i_bambino,n+1, i_bambino):
            # if i_piano % i_bambino ==0:
            # agisco solo sui piani 'multipli'
            if lista_secchi[i_piano]==0:
                lista_secchi[i_piano] = 1
```

```
else:
    lista_secchi[i_piano] = 0
return lista_secchi
```

Stima complessità computazionale

- Ho un ciclo esterno da 1 a n (l'iterazione k -esima coinvolge il bambino del piano k)
- Fissato l'indice del bambino, k , il bambino esamina tutti i piani da k a n con step di k . Esegue dunque $\frac{n}{k}$ passi. Per esempio, il bambino $k = 1$ esamina tutti i piani da 1 a n . Il bambino $k = 10$ esamina tutti i piani da 10 a n con step di 10, quindi ne esamina $\frac{n}{10}$
- il numero di passi elementari è dunque

$$T(n) = n + \frac{n}{2} + \frac{n}{3} + \dots + \frac{n}{k} + \dots + 1 = n \cdot \sum_{k=1}^n \frac{1}{k}$$

Se il bambino k -esimo esaminasse tutti i piani avremo $O(n^2)$. Ma ogni bambino esamina solo i piani multipli del suo, quindi la complessità si riduce molto.

In particolare, il valore

$$\sum_{k=1}^n \frac{1}{k}$$

è l' n -esimo *numero armonico*, che ha andamento asintotico¹

$$\sum_{k=1}^n \frac{1}{k} \sim \log(n)$$

(a meno di termini di ordine inferiore)

Dunque

$$T(n) = O(n \log(n))$$

¹https://en.wikipedia.org/wiki/Harmonic_number#Calculation

Bozza soluzione (c)

```
#include <stdio.h>
#include <stdlib.h>

int conta_secchi_pieni(int n) {

    int i, i_bambino, i_piano;
    int n_secchi_pieni;
    int *lista_secchi = malloc( (n+1) * sizeof(int)); // sizeof(*lista_secchi);

    // oppure staticamente, a seconda della versione C utilizzata
    // int lista_secchi[n + 1];

    for (i = 0; i <= n; ++i) {
        lista_secchi[i] = 0;
    }

    for (i_bambino = 1; i_bambino <= n; i_bambino++) {
        // Il bambino controlla il secchio al proprio piano e ai piani multipli
        i_piano = i_bambino;
        while (i_piano <= n) {
            if (lista_secchi[i_piano] == 0)
                lista_secchi[i_piano] = 1;
            else
                lista_secchi[i_piano] = 0;
            // Piano successivo da esaminare
            i_piano += i_bambino;
        }
    }

    // Conta quanti sono rimasti pieni (valore 1)
    n_secchi_pieni = 0;
    for ( i = 1; i <= n; i++) {
        n_secchi_pieni += lista_secchi[i];
    }

    free(lista_secchi);
    return n_secchi_pieni;
}

int main(void) {
    int n , pieni;
```

```
n= 100; // ad esempio, 100 piani
pieni = conta_secchi_pieni(n);
printf("Numero di secchi pieni per n = %d: %d\n", n, pieni);
return 0;
}
```

Ottimizzazione

Mettiamoci dal punto di vista del ‘secchio’: quante volte viene riempito/svuotato? Quante volte quanti sono i suoi divisori! Se il numero di divisori è pari allora lo stato del secchio non cambia. Se il numero di divisori è dispari allora lo stato del secchio cambia. Ci basta sapere se il numero dei divisori dell’indice del piano sono pari o dispari!

Per ogni n , se d è un divisore, anche n/d lo è. Quindi i divisori si raggruppano in coppie $(d, n/d)$.

Per esempio, 10 ha i seguenti divisori: (1,10); (2, 5)

Possiamo avere un divisore ‘non accoppiato’ solo se i divisori della coppia coincidono:

$$d = n/d \Rightarrow d^2 = n$$

cioè quando n è un quadrato perfetto.

Per esempio, 16 ha i seguenti divisori: (1,16); (2, 8), 4

per sapere quanti secchi saranno pieni al termine del processo è sufficiente contare il numero di quadrati perfetti

```
import math
def conta_secchi_pieni2(n):
    n_secchi_pieni = 0
    for i in range(1, n+1):
        r = int (math.sqrt(i))
        if i == r*r:
            # se è un quadrato perfetto
            n_secchi_pieni += 1
    return n_secchi_pieni
print(conta_secchi_pieni(10000))
```

Complessità computazionale $O(n)$

Per contare i quadrati perfetti abbiamo testato ciascun valore da 1 a n .

C'è però un metodo più rapido. I quadrati perfetti da 1 a n sono ovviamente in corrispondenza biunivoca con gli interi da 1 a $\lfloor \sqrt{n} \rfloor$.

Se ho $n = 100$ piani, gli interi da 1 a 10 generano i piani che sono quadrati perfetti: da $1^2 = 1$ a $10^2 = 100$. Quindi ho esattamente 10 quadrati perfetti.

```
import math
def conta_secchi_pieni3(n):
    return int(math.sqrt(n))
```

Complessità computazionale $O(1)$ [In realtà la complessità computazionale **dipende dal costo del calcolo della radice quadrata.**]